PlantPhoto_Flowise:
dal diagramma al codice · versione ampliata

Prof. Ing. Loris Penserini, PhD
Installazione, configurazione e manutenzione

Chatbot ibridi: LLM + Workflow · Parte 9

Dal diagramma al codice: ora leggiamo le decisioni

Le slide precedenti hanno mostrato l’architettura e il collaudo. Qui osserviamo come quelle scelte diventano istruzioni Python, endpoint HTTP, controlli di rete e query SQL.

Il codice non viene mostrato per impararlo a memoria: serve a riconoscere dove il sistema prende una decisione deterministica e dove, invece, delega una risposta al modello AI.

Domanda guida

Per ogni estratto chiediamoci: quale input riceve, quale controllo applica, quale stato modifica e quale componente può chiamare dopo?

La struttura del progetto assegna compiti precisi

app/ — API e controllo

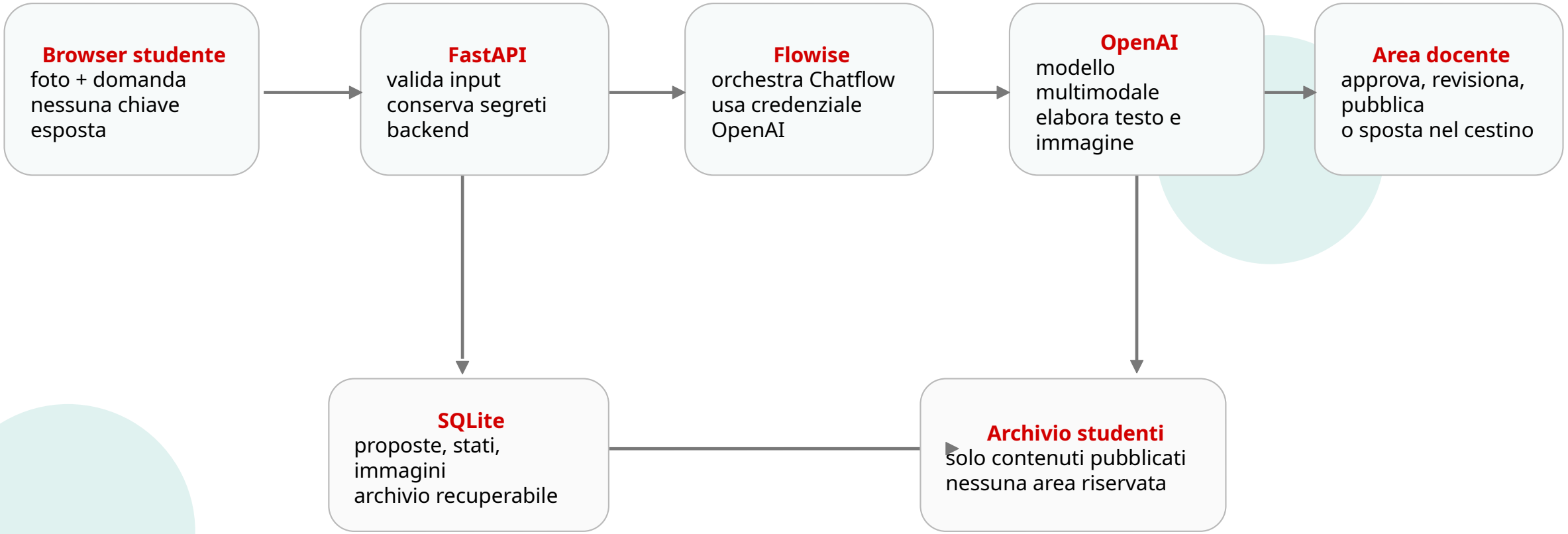
main.py costruisce l’app FastAPI. review.py definisce proposte, archivio e azioni docente. aula.py aggiunge il filtro per la modalità LAN.

services/ e web/ — lavoro specializzato

image_service.py controlla le fotografie; flowise_client.py comunica con Flowise; review_store.py conserva proposte e stati. web/ contiene le pagine HTML.

scripts/ prepara rete e firewall. data/ contiene archivio.sqlite3. I BAT offrono al docente due avvii distinti: locale e aula.

Una richiesta attraversa moduli distinti



Lo studente non raggiunge direttamente Flowise o OpenAI. FastAPI riceve la proposta, valida i dati e la salva. Solo l'approvazione docente attiva Flowise; la risposta torna in revisione e diventa pubblica dopo un'altra decisione umana.

Due avvii, due superfici di rete diverse

Avvio locale

Avvia_Locale.bat esegue Uvicorn su 127.0.0.1:8001. Il servizio accetta connessioni soltanto dal PC docente; è la modalità più semplice e sicura per sviluppo e revisione.

Avvio aula

Avvia_Aula.bat seleziona la scheda LAN, imposta IP e sottorete, prepara una regola firewall limitata e avvia Uvicorn su 0.0.0.0:8001. Il middleware applicativo continua a filtrare i client.

0.0.0.0 non è un URL da digitare: significa “ascolta sulle interfacce disponibili”. Gli studenti usano l’IP reale del PC docente, per esempio http://192.168.1.133:8001/.

aula.py controlla ogni richiesta remota

Il middleware viene eseguito prima dell'endpoint richiesto. Distingue il loopback del docente dai client LAN e limita questi ultimi alle sole funzioni pubbliche.

Codice reale

```
peer = request.client.host
local = ip_address(peer).is_loopback

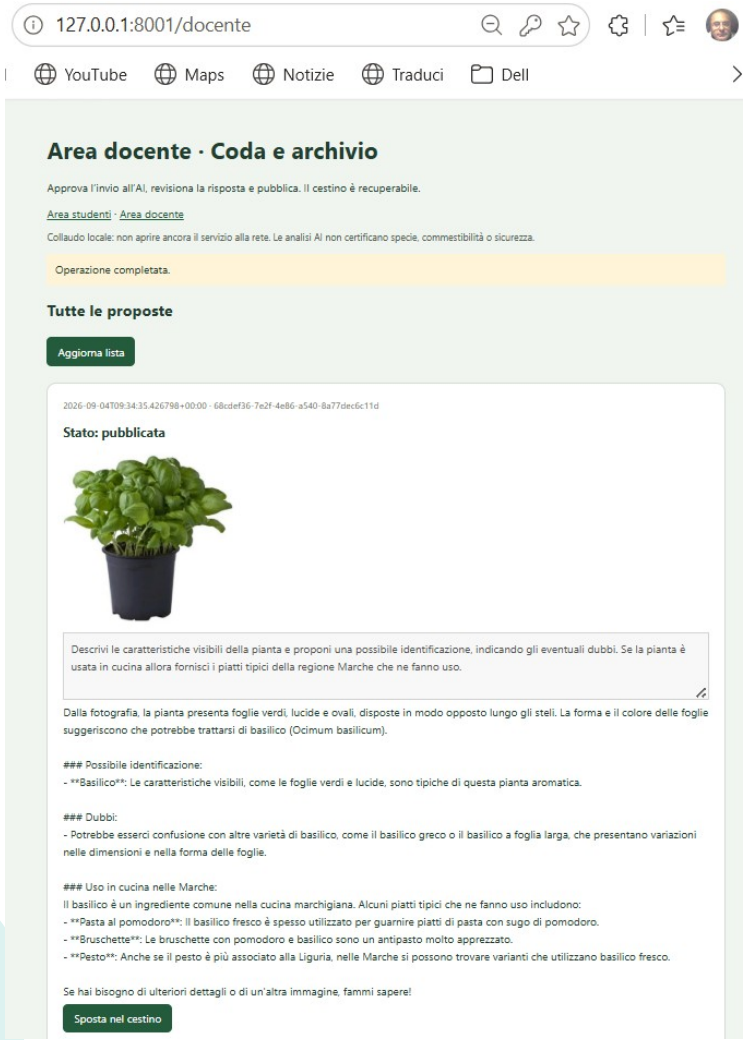
if not local:
    path = request.url.path
    public = (method == 'GET' and
              path in ('/', '/api/archivio'))
    public |= (method == 'POST' and
               path == '/api/proposte')
    if not public:
        return JSONResponse(..., 403)
```

Lettura riga per riga

request.client.host è l'IP osservato dal server. Il loopback identifica il PC docente. Per un client remoto viene costruita una lista positiva di metodi e percorsi consentiti: tutto ciò che non compare è rifiutato con HTTP 403.

Il controllo è server-side: nascondere un pulsante nell'HTML non sarebbe sufficiente. Anche una richiesta costruita manualmente deve rispettare le stesse regole.

Le tre pagine HTML servono utenti differenti



index.html è l'interfaccia studente: mostra l'archivio pubblicato e invia foto e domanda. review.html è la coda docente: approva l'AI, corregge, pubblica o sposta nel cestino. aula.html apre o chiude l'accesso e gestisce gli IP autorizzati.

Il browser non decide i privilegi

Le pagine invocano endpoint diversi, ma l'autorizzazione è verificata nuovamente da FastAPI. Un client remoto non può trasformarsi in docente cambiando URL o modificando JavaScript.

La GUI rende il processo comprensibile; il backend rende effettiva la separazione dei ruoli.

POST /api/proposte salva, ma non chiama l’AI

L’endpoint accetta un file e una domanda. Prima legge al massimo 5 MiB, poi valida realmente l’immagine e infine crea una proposta nello stato attesa.

Codice reale

```
@router.post('/api/proposte', status_code=201)
async def propose(request, file, question):
    data = await file.read(MAX_IMAGE_BYTES + 1)
    if len(data) > MAX_IMAGE_BYTES:
        raise HTTPException(413, 'Foto oltre 5 MiB')

    upload = await run_in_threadpool(prepare_image, data)
    ident = request.app.state.review_store.create(
        question.strip(), data, upload['mime']
    )
    return {'id': ident, 'status': 'attesa'}
```

Che cosa accade davvero

La fotografia viene memorizzata sul PC docente. prepare_image verifica il contenuto e ricava il MIME attendibile. ReviewStore assegna un UUID e registra lo stato iniziale. Nessuna funzione Flowise compare in questo percorso.

La separazione tra “ricevere una proposta” e “spendere una chiamata AI” inserisce consapevolmente il docente nel loop.

image_service.py non si fida del nome del file

Il browser può dichiarare un MIME errato o inviare un file rinominato. Pillow apre i byte reali, controlla formato, dimensioni, animazione e integrità.

Codice reale

```
formats = {'JPEG', 'PNG', 'WEBP'}

with Image.open(BytesIO(data)) as im:
    if im.format not in formats:
        raise ImageValidationError(...)
    if im.width * im.height > 20_000_000:
        raise ImageValidationError(...)
    if getattr(im, 'n_frames', 1) != 1:
        raise ImageValidationError(...)
    im.verify()

with Image.open(BytesIO(data)) as im:
    im.load()
```

Perché due aperture?

verify() controlla la struttura senza decodificare completamente tutti i formati. Il servizio riapre poi il flusso ed esegue load(), così intercetta anche immagini troncate o danneggiate che potrebbero superare un controllo superficiale.

La validazione è deterministica: decide se la fotografia può entrare nel workflow prima che il modello AI riceva qualunque dato.

flowise_client.py costruisce una sola richiesta autenticata

FastAPI non chiama OpenAI direttamente. Invia domanda e fotografia alla Prediction API del Chatflow configurato, con timeout espliciti e senza retry automatici.

Codice reale

```
headers = {
    'Authorization': 'Bearer ' + api_key,
    'Accept': 'application/json'
}
payload = {
    'question': question.strip(),
    'streaming': False, 'uploads': uploads
}

async with httpx.AsyncClient(
    timeout=timeout, follow_redirects=False,
    trust_env=False
) as client:
    response = await client.post(
        settings.prediction_url,
        headers=headers, json=payload
    )
```

Tre scelte intenzionali

follow_redirects=False evita di inoltrare credenziali verso destinazioni inattese. trust_env=False ignora proxy e credenziali implicite del sistema per questo client locale. I timeout separano connessione, lettura, scrittura e pool.

La chiave Flowise vive nel backend e viene inviata nell'header Authorization; il browser studente non la riceve mai.

Gli errori esterni vengono tradotti in messaggi controllati

Il client non restituisce al browser header, chiavi o corpi grezzi. Classifica gli errori più utili e conserva il dubbio quando una richiesta potrebbe essere stata elaborata.

Codice reale

```
except httpx.TimeoutException:
    raise FlowiseError(
        'Tempo superato: nessun retry automatico.'
    )
except httpx.ConnectError:
    raise FlowiseError(
        'Verificare Flowise, indirizzo e porta.'
    )

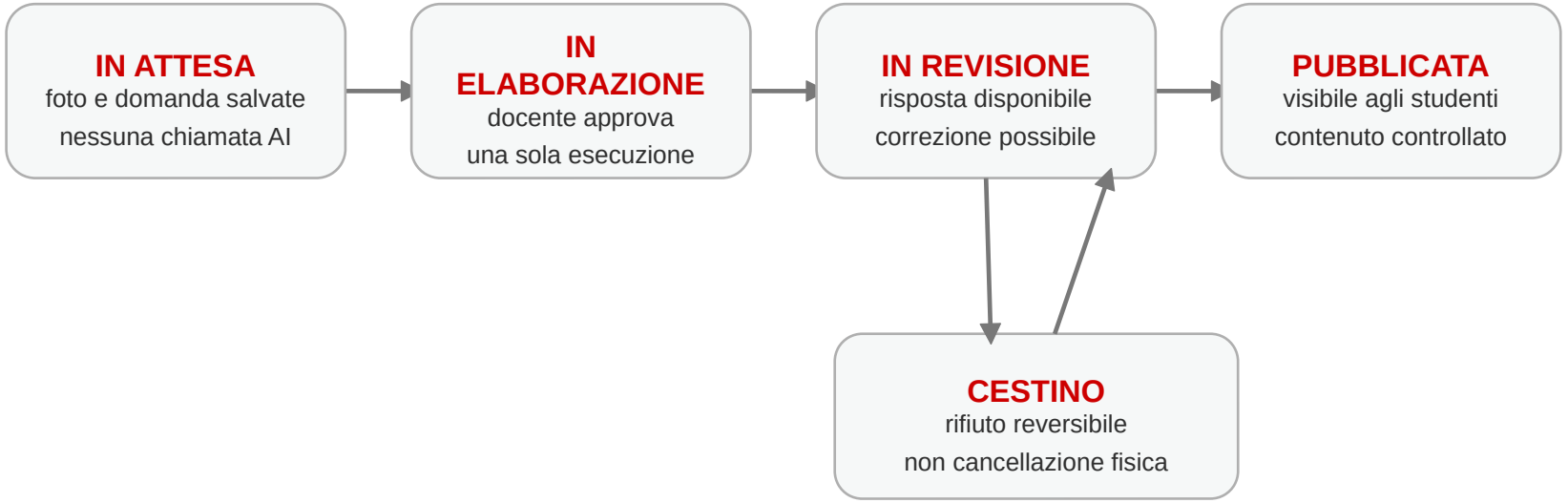
if not response.is_success:
    if 'invalid chatflow id' in body:
        detail = 'Verificare il Chatflow ID.'
    elif status in (401, 403):
        detail = 'Verificare chiave e associazione.'
    raise FlowiseError(f'HTTP {status}. {detail}')
```

Perché nessun retry automatico?

Dopo un timeout FastAPI non può sapere con certezza se Flowise o OpenAI abbiano già elaborato la richiesta. Ripeterla potrebbe duplicare il costo e produrre due risultati differenti. Il sistema passa quindi allo stato errore e richiede una verifica umana.

Un errore ben gestito non nasconde l’incertezza: la rende visibile e impedisce al software di agire due volte senza autorizzazione.

Gli stati impediscono operazioni fuori sequenza



Controllo implementato

Ogni azione specifica sia lo stato di partenza consentito sia quello di arrivo. Per esempio, l'invio all'AI è ammesso da attesa o errore; la pubblicazione è ammessa soltanto da revisione. Il cestino è reversibile.

ReviewStore aggiorna lo stato in modo condizionato

La funzione change non legge lo stato e poi lo aggiorna in due operazioni separate. Incorpora il controllo direttamente nella clausola WHERE dell'UPDATE.

Codice reale

```
def change(self, ident, allowed, target, **fields):
    assignments = 'status=?, ...'
    placeholders = ','.join('?' for _ in allowed)

    with self.connect() as db:
        cursor = db.execute(
            f'UPDATE proposals SET {assignments} '
            f'WHERE id=? AND status IN ({placeholders})',
            [target, *fields.values(), ident, *allowed]
        )
        return cursor.rowcount == 1
```

Che cosa garantisce

L'aggiornamento riesce soltanto se l'ID esiste e lo stato corrente appartiene ad allowed. Se un'altra richiesta ha già modificato la proposta, rowcount vale 0 e FastAPI restituisce HTTP 409: l'utente deve aggiornare la lista.

La condizione nel database riduce le corse tra richieste concorrenti e impedisce una transizione basata su informazioni ormai vecchie.

SQLite conserva proposta, foto e storia operativa

Una riga per proposta

La tabella proposals contiene UUID, data UTC, domanda, immagine BLOB, MIME, stato, risposta ed errore. Non serve un database esterno: il laboratorio resta autocontenuto sul PC docente.

Stato incerto dopo un arresto

Nel costruttore, ogni riga rimasta in elaborazione viene trasformata in errore. La risposta remota potrebbe essere stata generata, ma non ricevuta: il docente verifica Flowise prima di autorizzare un nuovo invio.

listing(public=True) aggiunge WHERE status='pubblicata'. Le fotografie non pubblicate sono servite soltanto dagli endpoint protetti dell’area docente.

Le chiavi risiedono in livelli differenti

Il file .env del backend contiene soltanto ciò che FastAPI deve conoscere. La chiave OpenAI rimane nella credenziale del nodo Flowise e non viene copiata nel progetto Python.

Codice reale

```
# .env sul PC docente
FLOWISE_BASE_URL=http://127.0.0.1:3000
FLOWISE_CHATFLOW_ID=<id-del-chatflow>
FLOWISE_API_KEY=<chiave-flowise>
TEACHER_PASSWORD=<password-docente>

# Dentro Flowise
Credenziale: OpenAI_Docente
OPENAI_API_KEY=<chiave-openai>
```

Non sono quattro sinonimi

FLOWISE_BASE_URL individua il servizio locale. FLOWISE_CHATFLOW_ID sceglie il workflow. FLOWISE_API_KEY autorizza il backend verso quel Chatflow. OPENAI_API_KEY consente a Flowise di usare il modello a pagamento. TEACHER_PASSWORD protegge l’area docente.

Il browser riceve soltanto HTML, JSON e fotografie autorizzate. Nessuna chiave deve comparire nel JavaScript, nell’URL o nelle schermate degli studenti.

Firewall e FastAPI applicano controlli complementari

Firewall Windows
consente TCP 8001 solo dal PC studente scelto

→

middleware aula.py
verifica IP, apertura aula e percorso richiesto

Il firewall decide se il pacchetto può raggiungere Uvicorn. Il middleware decide poi se quel client può usare la funzione richiesta. L'area docente resta limitata al loopback e protetta anche con autenticazione HTTP Basic.



Indirizzi da non confondere

127.0.0.1 indica il PC stesso. 0.0.0.0 è un binding, non una destinazione. L'IP LAN del docente è l'indirizzo usato dagli studenti. Flowise resta su 127.0.0.1:3000 e non viene esposto in aula.

Seguiamo una proposta dall’invio alla pubblicazione

1. Acquisizione e attesa

index.html invia POST /api/proposte. aula.py verifica l’IP; review.py valida testo e foto; ReviewStore salva stato=attesa. La risposta conferma che nessuna AI è stata chiamata.

2. AI, revisione e pubblicazione

Il docente invia: stato=elaborazione, chiamata Flowise, poi stato=revisione. Corregge il testo e pubblica: stato=pubblicata. Solo allora GET /api/archivio mostra il contenuto agli studenti.

Ogni passaggio lascia una traccia nello stato SQLite. Questo rende il workflow osservabile e permette di distinguere un errore di rete, un rifiuto di autorizzazione e un errore dell’AI.

Il codice mantiene l’AI dentro un processo controllato

Deterministico

FastAPI valida dati e utenti, decide quando l’AI può essere chiamata e applica le transizioni. SQLite conserva una storia locale e verificabile.

Probabilistico

Flowise orchestra il Chatflow e custodisce la credenziale OpenAI. Il modello analizza testo e immagine, ma produce una proposta che può contenere errori.

Supervisionato

Lo studente propone; il software controlla; il modello genera; il docente revisiona e pubblica. Il valore del chatbot ibrido nasce dalla cooperazione di codice deterministico, AI probabilistica e supervisione umana.